Bare-Metal vs. VM Malware Analysis

Author Name: Brooke Waltz
Professor: Dr. Mohammad Almalag

Executive Summary: Modern malware is known to detect when it is running in a virtual environment and hides its malicious behavior to seem innocent. This means free, VM-based sandbox tools used commonly by students and small IT teams may be missing real threats. This project tests whether that detection gap still exists with modern tools by running known evasive malware samples through both a modern VM sandbox (CAPE) and a bare-metal machine with affordable, consumer friendly hardware, then comparing what each setup actually catches. The end result will be a practical, low-cost standard that IT technicians, students, and small security teams can use to understand the limits of VM-based analysis without needing corporate infrastructure.

Table of Contents

Introduction....3
Project Background....3
Project Description....3
Proposed Solution....4
Development Approach....4
High Level Plan....4
Project Schedule....6
Work Breakdown Structure....6
Project Calendar....6
Intermediate Milestones....6
Intermediate Milestone #1 10/19....6
Intermediate Milestone #2 11/9....6
Weekly Updates....7
Weekly Update #1 9/23....7
Weekly Update #2 9/30....7
Weekly Update #3 10/7....7
Weekly Update #4 10/14....7
Weekly Update #5 10/21....7
Weekly Update #6 10/28....7
Weekly Update #7 11/4....7
Weekly Update #8 11/11....7
Weekly Update #9 11/18....8
Project Deliverables....8
Conclusion....8

Introduction

Modern malware often detects when it is being analyzed in a VM and either hides its malicious behavior or goes dormant, which means anyone testing malware on VMs can miss real threats. Bare-metal analysis solves this problem in theory, and researchers like Kirat, Vigna, and Kruegel demonstrated its effectiveness over a decade ago with their BareBox framework. Still, most free, accessible tools available today for students and small IT teams are still VM-based. This project will run known evasive modern malware samples through both a modern VM sandbox (CAPE) and an affordable bare-metal setup to see whether that same detection gap still holds true with today's tools and today's malware.

Project Background

This project matters because most small businesses and individual IT technicians do not have access to the kind of enterprise-grade malware analysis infrastructure that large security teams use. When a suspicious file shows up on a home network or a small business's systems, the go-to options are free VM-based sandbox or nothing at all. If VM-aware malware can reliably detect and evade those free tools, that is a real blind spot for the people least equipped to notice it.

I chose this project because it sits at the intersection of my current job as an IT technician and my long-term career goal of moving into network security or security engineering. At my job, I deal with suspicious files and systems that need to be assessed quickly, and I have relied on the assumption that sandbox results are trustworthy. When researching this topic, it made me realize that assumption does not always hold, and that the gap between lab-grade and budget malware analysis is a topic I want to understand firsthand.

There is paper that will be the foundation of this project. The 2011 BareBox paper written by Kirat, Vigna, and Kruegel proved that bare-metal analysis reveals malicious behavior that VM-aware malware hides from virtualized sandboxes, using 42 samples from seven malware families that stayed silent in virtual environments. That research used custom, purpose-built infrastructure that is not something a student or small IT team could reproduce realistically. This project proposes a specific, modern question: If a student uses only free, modern tools and a low-cost secondary PC, can they get results that still show this gap with malware?

Project Description

When this project is complete. It will produce a documented, reproducible comparison of malware behavior observed in a VM-based sandbox versus a bare-metal analysis environment, along with the tools used to log and compare that behavior. It is an applied security research project that combines existing open-source tools (CAPE Sandbox) with bare-metal monitoring and logging setup.

The intended audience for this project are IT technicians, cybersecurity students, and small security teams who want a realistic, budget-friendly answer to whether it is worth the extra time and hardware to run a bare-metal check on a suspicious file, rather than trusting a sandbox result by default. Within the broader field, this project fits applied malware analysis and detection engineering, specifically the field of sandbox evasions. It is meant to produce a practical reference rather than a new detection tool or piece of production software.

Proposed Solution

Development Approach

This project will use an iterative, research-driven approach rather than a software development lifecycle approach. The reason is that the core requirements here are not fixed feature specifications, they are research questions that depend on result I do not have until I am halfway through the project. For example, how many samples actually show behavioral difference between the environments, or how reliable my bare-metal monitoring setup turns out to be, can not be fully known in advance. An iterative approach lets me build and test the two environments individually first, validate that each one works correctly on its own, and then move into the comparison phase once both previous steps are stable, adjusting the scope (ex: sample count) based on how much time each phase actually take.

High Level Plan

The VM-based sandbox will run on a laptop using VMware Workstation as the hypervisor, with CAPE and the Windows 10 guest set up as sibling VMs rather than nested, since the laptop does not support nested virtualization. CAPE will run in an Ubuntu VM alongside a separate Windows 10 VM, both under VMware Workstation, with CAPE controlling the Windows guest directly over the virtual network using CAPE's built-in VMware Workstation machinery module rather than through a nested hypervisor. This sibling setup avoids the added timing anomalies and hypervisor artifacts that nested virtualization can introduce, though the overall setup will still be documented in detail (VMware version, guest tools status, and any other identifiable artifacts) since any hypervisor-based VM carries some baseline detectability that a genuinely bare-metal environment does not. The virtual LAN will use a host-only or internal network only, with no bridged or NAT adapter on either VM once initial setup and tool installation are complete, shared folders, clipboard, and drag-and-drop will be disabled on the Windows 10 guest, and the laptop's Wi-Fi will be physically disconnected during every run, verified beforehand with a harmless test file to confirm no traffic escapes the virtual LAN.

The bare-metal environment will be a Dell OptiPlex 7010, chosen for its age and legacy BIOS support, which avoids the firmware-level risks of newer UEFI-only systems. Before use, Intel AMT and Management Engine features will be disabled in the BIOS, since these allow out-of-band network access independent of the operating system and are a real containment concern for a machine that will run live malware. The machine will be checked for any Bluetooth card, including M.2 slots, and any found will be physically removed rather than only disabled in software, since a software toggle could be reversed by privileged malware. A known-good BIOS image will be saved offline ahead of time in case firmware recovery is ever needed. The OptiPlex's CPU, RAM, and disk size will be documented and roughly matched on the VM side, so hardware differences do not become a puzzling variable in the comparison.

To keep the comparison valid, both environments will run the same monitoring tool set, Procmon, Regshot, FakeNet-NG, and Autoruns, rather than relying on CAPE's built-in reports as the only VM-side data source; CAPE's reports will be used as a secondary cross-check instead. FakeNet-NG will run identically configured on both sides, so differences in simulated network responses do not get mistaken for genuine environmental differences. Before any real sample is

run, both environments will be calibrated using VM-detection tools such as Pafish and al-khaser to confirm exactly if there are any artifacts the VM setup leaks and that the bare-metal box reads as clean, establishing a baseline for use in later results.

Each sample will be run once in each environment. Because malware behavior can be non-deterministic (random filenames, mutations, timing), any sample that diverges between environments will be re-run two more times in both environments, and a divergence will only be recorded if it is reproducible across runs. Samples that do not diverge on the first pass will not be repeated, which is a stated limitation: intermittent differences in those samples could go undetected. Every run uses the same fixed duration of 45 minutes in both environments, since evasive samples often sleep rather than visibly detecting anything. Comparison is structured around a primary indicator: whether the sample runs to completion, goes dormant, exits early, or crashes, as defined in the methodology: crashed, dormant, exited early, or completed. File, registry, network, and persistence differences are supporting evidence rather than independently weighted criteria, since an early exit will naturally produce differences in all of them. A sample is logged as a behavioral difference when the primary indicator differs reproducibly, or when the primary indicator matches but two or more supporting categories differ. Samples tagged as VM-aware will be checked with VirusTotal to confirm they contain anti-VM capabilities rather than relying on the tag alone, with the runs themselves testing whether those capabilities trigger. A few non-evasive control samples (counted separately from the 8 to 10 target) will be included for comparison. Because many public samples are inactive or need specific arguments, roughly 15 candidates will be vetted up front, and samples will be discarded only under criteria defined before testing begins.

The bare-metal machine remains physically disconnected from any real network at all times. Simulated connectivity comes from FakeNet-NG running in single-host mode over an unmanaged cheap switch that has nothing else connected to it, and any Bluetooth card is physically removed. Samples are kept in a password-protected archive until the moment of use and are transferred on a dedicated write-protected labelled USB drive used for nothing else. Each sample's SHA-256 hash is verified immediately before every run, not only when first logged, to catch mix-ups or corruption early. The sample USB is removed once the sample is on the machine. The Clonezilla image drive and boot USB are physically unplugged during every sample run, not just left idle, since wipers and ransomware can target any attached disk. The machine is re-imaged from a checksum-verified Clonezilla image after every sample, whether the environment looks clean or not. After each restore, and before the restored system is first booted, the restored partitions are hash-verified against the clean baseline recorded when the Clonezilla image was created, to confirm the restore itself was not corrupted. Next are logs, they are the one thing that must leave the bare-metal machine, so this path is treated as part of containment. At the end of each run's fixed duration, the final Regshot and Autoruns captures are taken, the FakeNet-NG and Procmon captures are stopped, and the Procmon output is exported to CSV inside the Windows install, since Procmon is the only tool that needs Windows to read its own logs. The machine is then shut down and booted from a Linux live labelled USB, where the Windows disk is mounted read-only and only text-based logs are copied to a dedicated log-out labelled USB drive. This drive is separate from the sample-transfer USB, is

formatted FAT32 or exFAT, never receives executables, scripts, or shortcuts, and is treated as unsafe. It is attached only to a dedicated intake VM in VMware Workstation, separate from the CAPE VM, with no network adapter, shared folders, clipboard, or drag-and-drop, and with AutoPlay disabled on the host, so the host never mounts it. Nothing from this path is ever opened directly on my main computer. Only sanitized results (comparison tables and short excerpts) go into the GitHub repository. Because logs come from a compromised OS, they are treated as untrusted and may be incomplete or tampered with, which is noted as a limitation. The sample-transfer and log-out drives are destroyed at the end of the project.

Project Schedule

Work Breakdown Structure

- Environment setup: build and validate the CAPE VM sandbox and the isolated bare-metal machine, including network isolation for both (estimated 2 weeks)
- Sample Sourcing and baseline documentation: select 8-10 samples (8 minimum) across 3-5 malware families from MalwareBazaar, focusing on documented VM-aware/evasive behavior, and record that known behavior as a control/baseline (estimated 1 week)
- Bare-metal monitoring/logging build: implement or configure tools to capture files, processes, and network activities on this machine without getting caught by the malware (estimated 2 weeks)
- Sample execution and data collection: run each sample through both environments, documenting results and re-imaging the bare-metal machine between runs (estimated 3 weeks)
- Comparison, analysis, and paper: compare VM vs. bare-metal results side by side, identify where behavior diverged, and document findings against the BareBox baseline (estimated 2 weeks)

Project Calendar

Intermediate Milestone #1 will be reached once both analysis environments (VM sandbox and bare-metal) are fully built, isolated, and validated using a single non-malicious test file to confirm logging works correctly on both sides. Intermediate Milestone #2 will be reached once sample execution is complete for at least the minimum target of 8 samples, with raw logs collected and ready for comparison analysis.

Intermediate Milestones

Intermediate Milestone #1 10/19

- Both environments (CAPE VM sandbox and isolated bare-metal machine) are built, network-isolated, contained and validated using a test run, confirming that monitoring and logging function correctly on each side before any real malware samples are introduced.

Intermediate Milestone #2 11/9

- Execution is complete for at least 8 malware samples across both environments, with full logs collected for each run and the bare-metal machine successfully re-imaged between samples, ready to move into comparison and analysis.

Weekly Updates

Weekly Update #1 9/23

- Set up CAPE Sandbox in an isolated VM on personal machine (2 hours)
- Source and prepare secondary PC hardware for bare-metal environment (2 days)
- Confirm both environments have no access to the home network (1 hour)

Weekly Update #2 9/30

- Validate CAPE sandbox with a known-safe test file (2 hours)
- Install Windows 10 (2 hours)
- Lockdown the OtiPlex: disable AMT in MEBx, disable PXE and Wake-on-LAN, check for a Bluetooth care, and save the Dell BIOS offline. (2 hours)
- Install Procmon, Regshot, FakeNet-NG, and Autoruns on both environments (3 hours)
- Recheck VMnet isolation (1 hour)

Weekly Update #3 10/7

- Clonezilla Optiplex's image with a checksum (4 hours)
- Containment tests with harmless files on both sides. (2 hours)
- Pafish and al-khaser calibration on both. (3 hours)
- Source a write-protected USB (1 hour)

Weekly Update #4 10/14

- Find 10 samples from MalwareBazar and document what is what (4 hours)
- Samples are downloaded inside a disposable, network-limited VM, hashed, put in a password-protected archive, and then copied to the USB. (2 hours)
- Make a per-run checklist, pre-run checklist, and comparison template (2 hours)
- Do a full cycle with a harmless file with restore and log extraction (3 hours)

Weekly Update #5 10/21

- Run all 8 samples in CAPE with the in-guest tools running too, with reverting to a clean snapshot between each. Then record the fixed run duration (6 hours)
- Capture some footage (1 hour)
- Start comparison notes for each sample (1 hour)

Weekly Update #6 10/28

- Run samples 1-4 on the OptiPlex, with a hash check, restore, and verification for each(5 hours)
- Extract logs and update the comparison notes (2 hours)

Weekly Update #7 11/4

- Run samples 5-8 on the OptiPlex (5 hours)
- Repeat runs on any samples that diverged, in both environments (4 hours)
- Swap in a spare for any dead sample (1 hour)

Weekly Update #8 11/11

- Complete the comparison tables (2 hours)
- Classify each samples: completion state, files, registry, network and persistence(2 hours)
- Analyze which samples went dormant or changed behavior, and relate the findings to BareBox (2 hours)
- Draft a paper (2 hours)

Weekly Update #9 11/18

- Finalize my paper connecting findings back to the BareBox research baseline (4 hours)
- Make the GitHub repository, log files, and documentation for final submission (4 hours)
- Build the slide deck with recordings. (2 hours)

Project Deliverables

Documentation

- System Vision Document: 1-2 page description of the project's purpose and goals
- Stakeholder Identification: IT Technicians, cybersecurity students, and small security teams evaluating whether free VM-based sandboxes are sufficient for their needs
- Analysis Results/User Stories: side-by-side behavioral comparison of each malware sample across the VM sandbox and bare-metal environment, including which samples showed evasive or dormant behavior in one environment but not the other
- Architecture Diagrams: visual representation of both environments, network isolation setup, and the data/logging flow between them

Technical Deliverables

- Zip Archive containing all logging/monitoring tools and configuration used for the bare-metal environment
- Link to GIT repo with all code, sanitized logs, and documentation
- Instructions/readme document explaining how the instructor can review or replicate the environment setup and results.

Final Presentation Deliverables

- A short slide deck (10–15 minutes) walking through the problem, the two environments built, and the comparison results
- A recorded demo showing one malware sample run through both the VM sandbox and the bare-metal setup, with the logged behavior shown side by side
- A summary results table/chart showing, across all tested samples, how many showed different behavior between the two environments
- A brief discussion of limitations and what a larger-scale version of this project (more samples, more families) could show in the future

Conclusion

This project will build and compare two malware analysis environments, a modern free VM sandbox and a low-cost bare-metal setup, to test whether the sandbox evasion gap identified in prior academic research still holds true using today's tools and today's malware. It is intended for IT technicians, students, and small security teams who do not have access to expensive infrastructure and need a realistic sense of what free tools can and can not catch. The project is expected to take the full semester, split building and validating both environments, running the samples, and analyzing the results.

The main area of concern is time management around the bare-metal re-imaging process, since each sample run requires restoring the machine to a clean state before the next test. This can slow progress if the process takes longer than expected, especially if it needs removal from BIOS

(extreme cases: new hardware). Another risk is malware detecting the custom logging tools on the bare-metal side. If this occurs, the logging approach will need to be revised to be less detectable.